



# O que é Formik?

- Biblioteca popular para gerenciamento de formulários em React.
- Elimina a necessidade de controlar estado, validar campos e tratar submissão manualmente.

 **Sem Formik**

- Controlar cada campo no state
- Lógica de validação manual
- Tratar erros individualmente
- Código verboso e repetitivo

 **Com Formik**

- Estado centralizado
- Validação declarativa
- Erros automáticos por campo
- API simples e consistente

# O que é Yup?

- Yup é uma biblioteca de validação de schema em JavaScript. Permite definir regras de validação de forma declarativa e legível, retornando mensagens de erro personalizadas.



## Schema-based

Define regras como objetos JavaScript



## Encadeável

API fluente com métodos encadeados



## i18n Friendly

Mensagens de erro customizáveis

# Instalação

- `npm install formik yup`
- Instalar types: `npm install -D @types/yup`

# Configuração — useFormik Hook

useFormik — exemplo básico

```
const formik = useFormik({
  initialValues: {
    nome: 'abc',
    email: '',
  },
  onSubmit: async (values) => {
    await enviarDados(values)
  },
})

return (
  <form onSubmit={formik.handleSubmit}>
    <input
      name="nome"
      onChange={formik.handleChange}
      value={formik.values.nome}
    />
  </form>
)
```

## Propriedades principais

### initialValues

Valores iniciais de cada campo do formulário

### onSubmit

Função executada ao submeter com sucesso

### handleChange

Handler genérico para onChange dos inputs

### handleSubmit

Handler para o evento onSubmit do form

### values

Objeto com os valores atuais dos campos

### errors

Objeto com os erros de validação

### touched

Campos que o usuário interagiu

# Validação com Yup

validationSchema com Yup

```
import * as Yup from 'yup'

const schema = Yup.object({
  nome: Yup.string()
    .min(3, 'Mín. 3 chars')
    .required('Nome é obrigatório'),
  email: Yup.string()
    .email('Email inválido')
    .required('Email é obrigatório'),
  idade: Yup.number()
    .min(18, 'Mínimo 18 anos')
    .required('Informe sua idade'),
  senha: Yup.string()
    .min(8).required()
})

// Integrar no formik:
validationSchema: schema
```

## Validadores Yup

### string()

.email() .url() .uuid() .min() .max() .matches()

### number()

.min() .max() .positive() .integer() .truncate()

### boolean()

.isTrue() .isFalse() .default()

### array()

.of() .min() .max() .compact() .ensure()

### object()

.shape() .pick() .omit() .camelCase()

### date()

.min() .max() .nullable()

### .required()

Campo obrigatório (disponível em todos)

### .nullable()

Permite null como valor válido

```
8
9  export default function Login() {
10    interface FormValues {
11      username?: string;
12      password?: string;
13    }
14
15    // disponibiliza todas as funções necessárias para manipulação do formulário
16    const formik = useFormik<FormValues>({
17      // valores iniciais do formulario
18      initialValues: {
19        username: "",
20        password: "",
21      },
22      validationSchema: loginSchema,
23      // função disparada quando o formulário é enviado
24      onSubmit: (values) => {
25        console.log(values);
26      }
27    });
28
29    const {
30      handleSubmit,
31      values,
32      handleChange,
33      errors
34    } = formik;
```

```
35
36 return (
37   <>
38     <div className="mx-auto my-auto">
39       <Form onSubmit={handleSubmit}>
40         <InputText label="usuário:" inputName="username"
41           placeholder="Nome do usuário"
42           value={values.username}
43           id="username"
44           onChange={handleChange}
45           error={errors.username}
46         ></InputText>
47
48         <br/>
49         <label>senha:</label>
50         <input
51           type="password"
52           value={values.password}
53           onChange={handleChange}
54           id="password"
55         ></input>
56         <br/>
57         <br/>
58         <Button variant="primary" type="submit">entrar</Button>
59       </Form>
60     </div>
61   </>
62 );
63 }
64
```



src &gt; components &gt; InputText.tsx &gt; InputTextProps &gt; placeholder

```
1  import { ChangeEvent } from 'react';
2  import { Form } from 'react-bootstrap';
3
4  interface InputTextProps {
5    label: string;
6    inputName: string;
7    placeholder?: string;
8    value?: string;
9    id?: string;
10   onChange?: (e: ChangeEvent<any>) => void
11   error?: string | undefined;
12 }
13
14 // props: parametro da função,
15 // que recebe todas as propriedades do componente (através de um objeto)
16 export function InputText(props: InputTextProps) {
17   // const label = props.label;
18   const {
19     label, inputName, placeholder,
20     value, id, onChange, error, }
21     = props;
22
23   return (
24     <>
25       <Form.Label htmlFor={id}>{label}</Form.Label>
26       &nbsp;
27       <Form.Control id={id} type="text" name={inputName}
28         placeholder={placeholder} value={value}
29         onChange={onChange}
30         isValid={!error} ></Form.Control>
31
32       <Form.Control.Feedback type="invalid">
33         {error}
34       </Form.Control.Feedback>
35     </>
36   );
37 }
```



**OBRIGADO**